



شماره دانشجویی:

۸۱۰۱۰۱۴۹۲



تئوری زنجیره‌های بلوکی
نام و نام خانوادگی: علیرضا کریمی

تمرین شماره ۵

سوال ۱

(a)

تغییری که در *Proof of Stake Version 3* یا به اختصار *PoSV3* رخ داد، اضافه شدن ایده‌ی *Header-First* از بیت‌کوین بود.

در این ایده، به جای دانلود کل بلاک (که شامل تمام جزئیات تراکنش‌هاست)، نودها ابتدا *Block Headers* را دریافت می‌کنند تا یک سری چک اولیه انجام دهند. در صورتی که این چک‌ها موفقیت‌آمیز بود، نود منتظر دریافت محتوای کامل بلاک می‌ماند تا اعتبارسنجی نهایی را انجام دهد. این کار باعث می‌شود نودها سریع‌تر بتوانند بررسی کنند که آیا یک بلاک مربوط به *Longest Chain* است یا نه. همچنین اگر چندین بلاک‌هدر دریافت شود، می‌توانند با مقایسه هش بلاک قبلی در هر هدر، زنجیره‌ها را بررسی کنند.

در *Block Header* اطلاعاتی مانند هش بلاک قبلی، *Merkle Root* (که چون داده‌های کامل تراکنش‌ها هنوز موجود نیست، نمی‌توان درستی آن را چک کرد)، تایم‌استمپ و سایر اطلاعات پایه وجود دارد. بنابراین با داشتن فقط بلاک‌هدرها، امکان بررسی اینکه یک *Coin* آیا واقعاً *Unspent* است یا نه وجود ندارد.

در پروتکل *PoS*، بلاک‌های جدید توسط سهام‌گذاری روی سکه‌های *Unspent* تولید می‌شوند. حال، مهاجم قبل از اینکه سکه‌ای را *Spend* کند، یک *Fork* از زنجیره اصلی در بلاک‌های قبلی آن ایجاد می‌کند. از آنجایی که نود فقط بلاک‌هدر را دریافت کرده، دو چک اولیه انجام می‌دهد:

۱. آیا سکه‌ی استفاده‌شده در *Main Chain* وجود دارد؟
۲. آیا هش کرنل تولیدشده از فرمول زیر، کمتر از مقدار هدف است؟

$$\text{hash}(\text{prev-block-data}, \text{current-time}, \text{UTXO}) \leq \text{difficulty} \times \text{coins}(\text{UTXO}) \times \text{timeweight}(\text{UTXO})$$

هر دوی این چک‌ها فقط نسبت به *Main Chain* انجام می‌شوند و وضعیت خرج‌شده یا نشده‌بودن یک سکه در یک *Fork* در نظر گرفته نمی‌شود.

نکته‌ی مهم این است که:

• چک اول فقط بررسی می‌کند که آیا سکه در گذشته در زنجیره‌ی اصلی وجود داشته یا نه، اما بررسی نمی‌کند که آیا آن سکه *spent* شده یا نه.

• چک دوم نیز برای بررسی سختی استخراج بلاک است، اما چون ورودی‌های آن نیز ناقص هستند (بر اساس همان سکه‌هایی که ممکن است خرج شده باشند)، این چک نیز قابل اتکا نیست.

نود فقط بررسی می‌کند که آیا آن تراکنش در *Transaction Database* وجود دارد یا نه، اما به *UTXO Set* کامل (که شامل وضعیت خرج شده یا نشده سکه‌هاست) دسترسی ندارد. دلیل این موضوع استفاده از ایده‌ی *Header-First* و عدم دریافت کامل اطلاعات بلاک است.

در نتیجه، کافیست مهاجم یک *Fork* پس از تولید آن سکه و پیش از خرج شدن آن در زنجیره اصلی ایجاد کند. سپس آن سکه را در این *Fork* به صورت پی‌درپی به خودش منتقل کرده و بلاک‌هایی حاوی این تراکنش‌ها را ماین کند. فقط هدر این بلاک‌ها را برای نودهای دیگر ارسال می‌کند. اگرچه در واقعیت فقط آخرین تراکنش در این زنجیره دارای خروجی خرج نشده است، اما نودها چون نمی‌توانند وضعیت *Unspent* بودن را بررسی کنند و فقط وجود آن سکه در زنجیره اصلی را تأیید می‌کنند، کل این ساختار تقلبی را معتبر می‌پندارند.

مهاجم با این کار اصطلاحاً مقدار **Apparent Stake** یا «سهام ظاهری» خود را افزایش می‌دهد. یعنی تعداد زیادی خروجی سکه دارد که از نظر نودها همگی قابل سهام‌گذاری هستند، حتی اگر در واقع خرج شده باشند. این باعث می‌شود احتمال موفقیت مهاجم در ساخت بلاک‌های *PoS* بالا برود، بدون اینکه واقعاً آن قدر سرمایه در اختیار داشته باشد.

در نهایت، مهاجم طول *Fork* را از زنجیره اصلی نیز بیشتر می‌کند تا نودهای دیگر آن را به عنوان زنجیره غالب بپذیرند. چون نودها به دلیل نداشتن *UTXO Set* نمی‌توانند بررسی دقیق‌تری داشته باشند، سهام‌های تولیدشده از تمام این سکه‌ها را می‌پذیرند.

اگر نودها از ابتدا بلاک کامل را به جای فقط هدر دریافت می‌کردند و دسترسی به وضعیت واقعی خرج نشده یا نشده سکه‌ها داشتند، این حمله به طور کامل قابل شناسایی و خنثی‌سازی بود.

علاوه بر این، مهاجم می‌تواند با ارسال مداوم این بلاک‌هدرها، فضای دیسک نودها را پر کرده و باعث شود نود دچار *Crash* شده یا بخشی از اطلاعات خود را از دست بدهد. [۱]

(b)

بنابراین، حتی اگر مهاجم تنها 0.01% از کل سهم شبکه را در اختیار داشته باشد، می‌تواند با ساخت ۵۰۰۰ تراکنش در *Fork* ساخته‌شده، به صورت اسمی به سهمی معادل ۵۰ درصد شبکه دست پیدا کند. هر سکه را چندین بار خرج می‌کند و ظاهر فریبنده‌ای از سهام بالا می‌سازد. [۱]

(c)

این حمله نسبت به حالتی مشابه در بیت کوین بسیار کم‌هزینه‌تر است. چرا که در بیت کوین تولید هر بلاک نیاز به صرف زمان و توان محاسباتی بالا دارد، ولی در اینجا هزینه اصلی مهاجم فقط کارمزد تراکنش‌هایی است که به خودش می‌فرستد و بلاک‌هایی که با آن‌ها سهام تولید می‌کند، که بسیار ارزان‌تر است. [۱]

سوال ۲

الگوریتم *PoET (Proof of Elapsed Time)* که توسط شرکت *Intel* پیشنهاد شده، نسبت به *PoW* تأخیر کمتری در تأیید تراکنش‌ها دارد. این تفاوت به دلیل نحوه عملکرد این دو الگوریتم و نوع شبکه‌ای است که در آن‌ها به کار می‌روند.

در *PoW*، نودها برای حل یک معمای رمزنگاری با هم رقابت می‌کنند. زمان لازم برای پیدا کردن یک بلاک، از یک توزیع نمایی (*exponential distribution*) پیروی می‌کند، یعنی کاملاً تصادفی و غیرقابل پیش‌بینی است. در نتیجه ممکن است چند نود تقریباً همزمان بلاکی پیدا کنند و باعث ایجاد *fork* (انشعاب) شوند که حل این *fork* ها نیز باعث تأخیر در نهایی شدن تراکنش‌ها می‌شود.

اما در *PoET*، به هر نود یک زمان انتظار تصادفی (که از یک توزیع یکنواخت *uniform distribution* گرفته شده) توسط سخت‌افزار قابل اعتماد (مثل *Intel SGX*) داده می‌شود. نودی که کوتاه‌ترین زمان انتظار را دارد، بلاک بعدی را تولید می‌کند. چون این زمان‌ها قابل جعل نیستند و از قبل توسط همه قابل مشاهده‌اند:

- نیازی به رقابت محاسباتی نیست.
- فقط یک نود بلاک را پیشنهاد می‌دهد و در نتیجه *fork* طبیعی اتفاق نمی‌افتد.
- زمان تولید بلاک‌ها منظم‌تر و قابل پیش‌بینی‌تر است.
- و تأخیر تأیید تراکنش‌ها به حداقل می‌رسد.

همچنین *PoET* معمولاً در شبکه‌های *permissioned* (مجوزدار) استفاده می‌شود، یعنی شرکت‌کننده‌ها از قبل شناخته شده و قابل اعتماد هستند. در این شرایط، نیازی به دفاع در برابر *Sybil attack* (که در آن یک فرد سعی می‌کند با ایجاد نودهای جعلی کنترل شبکه را به دست گیرد) وجود ندارد. این اجازه می‌دهد که *PoET* از مکانیسم‌های اجماع سبک‌تر و سریع‌تری استفاده کند، بدون اینکه امنیت کاهش یابد. این محیط کنترل شده همچنین ارتباطات بین نودها را بهینه می‌کند و مقیاس‌پذیری (*scalability*) را افزایش می‌دهد.

درواقع در این روش، ما به جای *trust* بین شرکت‌کننده‌ها، به شبکه و *hardware* اینتل، *trust* می‌کنیم. دلیل اینکه در *PoET* می‌توان به زمان‌های انتخاب شده توسط نودها اعتماد کرد، استفاده از تکنولوژی *Intel SGX* است. این تکنولوژی یک محیط اجرای امن (*Trusted Execution Environment*) فراهم می‌کند که در آن:

- کد مربوط به تولید زمان انتظار در حافظه‌ای ایزوله (*enclave*) اجرا می‌شود.
- حتی سیستم‌عامل یا کاربر محلی نمی‌تواند به محتوای آن دسترسی داشته باشد یا آن را تغییر دهد.
- این سخت‌افزار می‌تواند به صورت رمزنگاری شده اثبات کند که زمان انتخاب شده منصفانه و طبق الگوریتم مشخصی تولید شده است (به این ویژگی *remote attestation* می‌گویند).
- بنابراین، در *PoET* ما به جای هزینه کردن انرژی بالا برای رسیدن به اجماع (مثل *PoW*)، به سخت‌افزار مطمئن اینتل اعتماد می‌کنیم که زمان انتظار را درست و بدون تقلب تولید کرده است. البته این مدل اعتماد، فرض می‌کند که خود *Intel* و سخت‌افزارش قابل اعتماد هستند و هیچ آسیب‌پذیری عمدی یا ناخواسته‌ای وجود ندارد.
- خلاصه: تأخیر کمتر در تأیید تراکنش‌ها در *PoET* به دلایل زیر است:
- حذف رقابت محاسباتی و مصرف انرژی بالا (برخلاف *PoW*) و در نتیجه *energy – saving* و سازگارتر با محیط زیست
- استفاده از توزیع یکنواخت برای زمان‌های انتظار، به جای توزیع نمایی
- عدم وجود *fork* طبیعی به دلیل انتخاب تعیین شده‌ی نود تولیدکننده بلاک
- اجرای آن در شبکه‌های *permissioned* با هماهنگی سریع و کارآمد و بدون نیاز به *trust* به شرکت‌کننده‌های دیگر
- اعتماد به سخت‌افزار امن *Intel SGX* که از طریق محیط اجرای ایزوله و اثبات رمزنگاری شده، اطمینان از منصفانه بودن انتخاب‌ها را فراهم می‌کند
- این ویژگی‌ها باعث می‌شوند *PoET* در مقایسه با *PoW*، هم سریع‌تر باشد و هم سازگارتر با محیط زیست.

سوال ۳

(a)

همانطور که در شکل سوال ۱.۳ مشخص است، به سوالات زیر پاسخ می‌دهیم:

در نمودار (هر یال نشان‌دهنده عمل “تأیید” یک تراکنش توسط تراکنش دیگری است و جهت یال از تأییدگر به تأییدشده) داریم:

۱.

به صورت کلی *tip* ها آن گره‌هایی هستند که هنوز هیچ کس آن‌ها را تأیید نکرده (یال خروجی از آن‌ها ندارند). در این تنگ، تراکنش‌های شماره: ۹، ۱۰ و ۱۱ هیچ تأییدگر جدیدی ندارند و پس بنابراین *tips* هستند.

۲.

۱.۲. عمق ($Depth$)

عمق یک تراکنش برابر است با طول (تعداد یال‌ها) طولانی‌ترین مسیر از «genesis» (تراکنش مبنا) تا آن تراکنش. مسیر طولانی‌ترین تا تراکنش ۴:

$$0 \rightarrow 2 \rightarrow 4$$

$$Depth(4) = 2 \iff 2 = \text{تعداد یال‌ها}$$

۲.۲. ارتفاع ($Height$) ارتفاع یک تراکنش برابر است با طول (تعداد یال‌ها) طولانی‌ترین مسیر از خود آن تراکنش تا یکی از $tips$. مسیرهای ممکن از ۴ تا $tips$:

$$4 \rightarrow 6 \rightarrow 9$$

$$4 \rightarrow 7 \rightarrow 10$$

$$4 \rightarrow 8 \rightarrow 9$$

$$4 \rightarrow 8 \rightarrow 11$$

$$Height(4) = 2 \iff 2 \text{ یال است بلندترین مسیر}$$

۳.۲. وزن تجمعی ($Cumulative Weight$) وزن تجمعی یک تراکنش برابر است با «وزن خود آن» (اینجا همه وزن ۱ دارند) + «تعداد همه تراکنش‌هایی که به صورت مستقیم یا غیرمستقیم آن را تأیید کرده‌اند». تراکنش‌هایی که (مستقیم یا با زنجیره‌ای از تأیید) تراکنش ۴ را تأیید می‌کنند:

$$6, 7, 8, 9, 10, 11$$

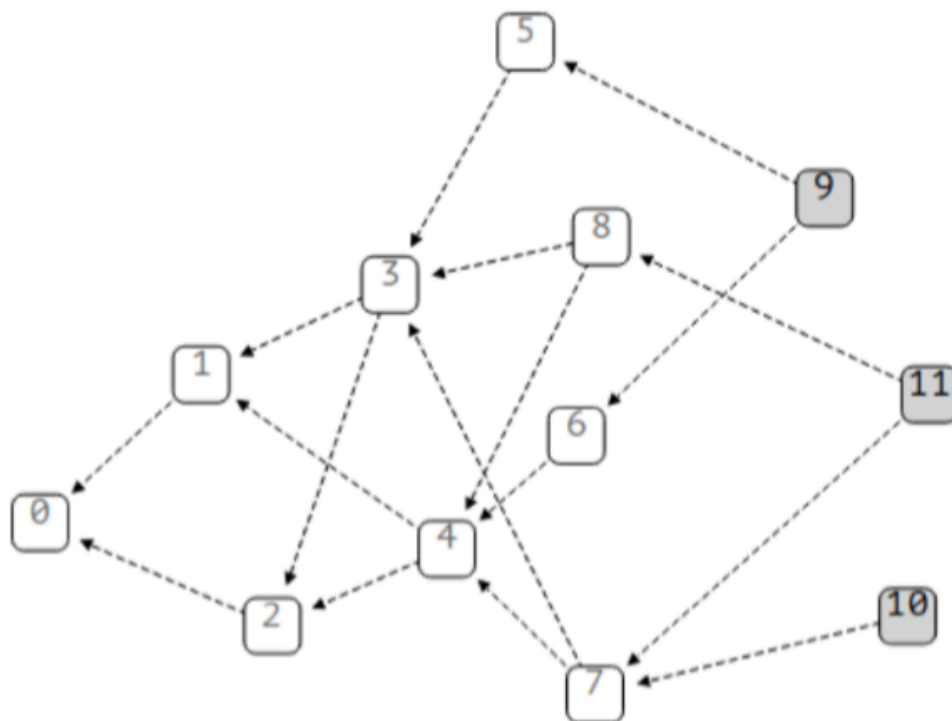
(۶ و ۷ و ۸ مستقیم، سپس ۹ از طریق ۶ یا ۸، و ۱۰ از طریق ۷، و ۱۱ از طریق ۸)

$$Cumulative Weight(4) = 1 + 6 = 7 \iff \text{جمعاً ۶ تراکنش به علاوه وزن خود ۴ (که ۱ است)}$$

[۲]

(b)

طبق این منبع [۲]، مسئله اصلی در استفاده از الگوریتم URW ، وقوع «نوک‌های تنبل» ($lazy tips$) است. نوک‌های تنبل به آن دسته از تراکنش‌های نوک (tip) گفته می‌شود که به جای تلاش فعالانه برای تأیید تراکنش‌های جدید، برخی از تراکنش‌های قدیمی را برای تأیید انتخاب می‌کنند. ما آن‌ها را «تنبل» می‌نامیم زیرا برای به‌روزرسانی خود



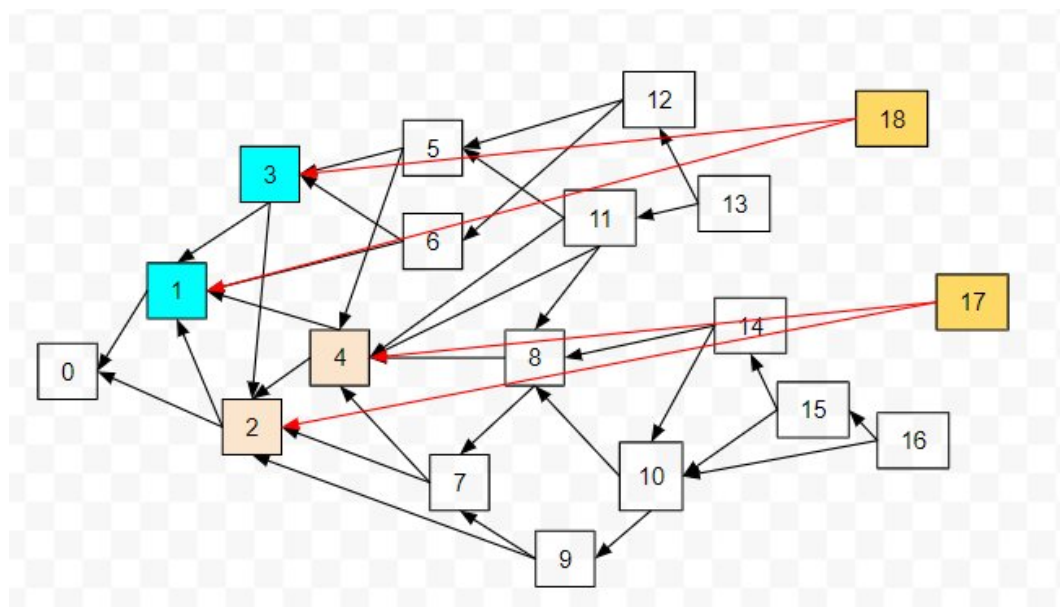
شکل سوال ۱.۳

با آخرین وضعیت شبکه *tangle* آیوتا (*IOTA Tangle*) تلاشی نمی‌کنند. در عوض، آن‌ها ترجیح می‌دهند تراکنش خود را بر اساس داده‌های قدیمی منتشر کنند. بیا بید نمودار سوال ۲.۳ را بررسی کنیم.

با ارجاع به نمودار سوال ۲.۳، تراکنش ۱۷ و تراکنش ۱۸ نوک تنبل (*lazy tips*) هستند، زیرا همانطور که به وضوح در نمودار نشان داده شده است، آن‌ها فقط تعدادی از تراکنش‌های قدیمی را تأیید کرده‌اند. اگر ما از روش انتخاب گام تصادفی غیروزی (*unweighted random walk*) استفاده کنیم، این دو نوک، شانس مساوی برای تأیید شدن با سایر نوک‌ها را خواهند داشت. رفتار آنها حتی توسط سیستم آیوتا جریمه نیز نخواهد شد. اگر تعداد زیادی نوک تنبل در سیستم وجود داشته باشد، شبکه تنگل (*Tangle*) راکد شده یا به سادگی از کار خواهد افتاد.

چگونه باید بر پدیده «نوک‌های تنبل» غلبه کنیم؟ ما می‌توانیم قانونی را تحمیل کنیم که تمام تراکنش‌های ورودی را ملزم به تأیید تراکنش‌های اخیر کند. با این حال، این روش اجباری با اصول عدم تمرکز و دموکراسی، که جوهره اصلی فناوری‌های دفتر کل توزیع شده است، در تضاد است. بنابراین، آیوتا روش اقناعی را انتخاب کرد. این روش یک سیستم تشویقی داخلی را پیاده‌سازی می‌کند که به شرکت‌کنندگان برای تأیید تراکنش‌های اخیر پاداش می‌دهد. این روش، (*Weighted Random Walk*) نامیده می‌شود.

گام تصادفی وزنی از استراتژی جهت‌دار (*bias strategy*) برای انتخاب مسیر به سمت یک نوک خاص استفاده می‌کند. استراتژی جهت‌دار شامل انتخاب مسیری است که بیشترین تعداد تراکنش تأیید شده را دارد، در مقابل مسیری که



شکل، سوال، ۲.۳: *Lazy Tips*

تعداد بسیار کمی تراکنش تأیید شده دارد. در اینجا ما اصطلاحی به نام «وزن تجمعی» (*cumulative weight*) را برای نشان دادن تعداد تأییدکنندگان تراکنش‌ها معرفی می‌کنیم. این بدان معناست که هرچه تعداد تأییدکنندگان بیشتر باشد، وزن تجمعی بالاتر است و بالعکس. تأییدکنندگان می‌توانند مستقیم یا غیرمستقیم باشند.

با این حال، حتی روش گام تصادفی وزنی نیز نقطه ضعف خود را دارد. اگر اصرار کنیم که هر تأیید تراکنش باید به شدت از این قانون پیروی کند، تصادفی بودن را به طور کامل از بین خواهیم برد و بسیاری از نوک‌ها بدون تأیید باقی خواهند ماند. سیستم تنگل در نهایت با تعداد زیادی تراکنش تأیید نشده یا یتیم (*orphan*) مواجه خواهد شد.

برای غلبه بر این مسائل، آیوتا پارامتری به نام α (آلفا) را برای کنترل «وزن» در گام تصادفی وزنی معرفی کرد. اگر $\alpha = 0$ باشد، این روش به گام تصادفی غیروزشی تبدیل می‌شود و مشکل تأیید تعداد زیادی نوک تنبل را ایجاد می‌کند. از سوی دیگر، اگر مقدار α را بیش از حد بالا تعیین کنیم، به یک گام تصادفی فوق وزنی منجر می‌شود که یتیم‌های زیادی را در تنگل به جا می‌گذارد.

بنابراین، یافتن یک تعادل خوب بین تأیید تعداد زیادی نوک تنبل و به جا نگذاشتن تعداد زیادی نوک یتیم، حائز اهمیت است. به طور خلاصه، یافتن یک مقدار ایده‌آل برای α برای حفظ انسجام اکوسیستم تنگل که آیوتا از آن پشتیبانی می‌کند، مهم است.

سوال ۴

(a)

وضعیت داده‌شده به صورت یک بازی استراتژیک دو نفره مدل‌سازی می‌شود. جدول سود (*Payoff*) برای دو ماینر به شکل زیر اصلاح شده است:

	ماینر <i>B</i> : ضعیف	ماینر <i>B</i> : قوی
ماینر <i>A</i> : ضعیف	(۵, ۵)	(۷, ۲)
ماینر <i>A</i> : قوی	(۲, ۷)	(۳, ۳)

تحلیل:

- اگر ماینر *B* گزینه «ضعیف» را انتخاب کند، ماینر *A* با انتخاب «قوی» سود ۷ می‌برد که بیشتر از ۵ است.
- اگر *B* گزینه «قوی» را انتخاب کند، *A* همچنان ترجیح می‌دهد گزینه «قوی» را انتخاب کند چون $۳ < ۲$.
- پس «قوی» راهبرد غالب برای ماینر *A* است.
- برای ماینر *B* نیز همین منطق برقرار است، بنابراین راهبرد غالب او نیز «قوی» است.
- در نتیجه، تعادل نش برابر است با (قوی، قوی) که سود آن برای هر دو ماینر (۳، ۳) است. این تنها وضعیت پایدار بازی است، چون هیچ کدام نمی‌توانند به تنهایی وضعیت بهتری برای خود ایجاد کنند.
- اما توجه داشته باشید که گزینه (ضعیف، ضعیف) سود (۵، ۵) را فراهم می‌کند که از نظر جمعی بهتر است، اما چون هر ماینر می‌تواند به تنهایی و فارق از انتخاب دیگری، با انتخاب «قوی» به سود بالاتری برسد، این وضعیت ناپایدار است.

(b)

- با وجود اینکه حالت (۵، ۵) از نظر جمعی بهتر است، ماینرها همچنان گزینه «قوی» را انتخاب می‌کنند چون:
- انتخاب «قوی» در برابر «ضعیف» باعث دریافت سود ۷ می‌شود.
 - اگر یکی «ضعیف» انتخاب کند و دیگری «قوی»، نفر ضعیف فقط ۲ می‌گیرد که بدترین حالت است.
 - بنابراین، از ترس فریب خوردن یا از دست دادن سود حداکثری، هر دو ماینر گزینه «قوی» را انتخاب می‌کنند.
 - منطق فردی منجر به انتخاب وضعیت (۳، ۳) می‌شود، در حالی که (۵، ۵) از نظر جمعی سودآورتر است.

(c)

برای هدایت ماینرها به سوی رفتارهای مشارکتی و کاهش رقابت مخرب، می‌توان از راهکارهای زیر در طراحی مکانیزم اجماع استفاده کرد:

۱. گواه اثبات سهام (*Proof of Stake – PoS*): انتخاب ماینر بر اساس میزان سرمایه به جای قدرت پردازشی انجام می‌شود. در نتیجه، انگیزه برای افزایش بی‌رویه توان پردازشی کاهش می‌یابد. مثال‌ها: *Ethereum 2.0*، *Tezos*، *Cardano*
۲. تنظیم پاداش استخراج: با اعمال مالیات تصاعدی یا بازدهی نزولی، سود ماینرهایی که توان پردازشی بالایی دارند کاهش می‌یابد. این امر انگیزه رقابت بیش از حد را کم می‌کند.
۳. جریمه و اعتبارسنجی: رفتارهای مضر مانند فورک‌های مکرر یا استخراج خودخواهانه می‌توانند با جریمه همراه باشند. مثال: مکانیزم *Slashing* در شبکه *Ethereum PoS*
۴. پروتکل‌های مشارکتی: پروتکل‌هایی نظیر *Mina Protocol*، *Fair PoW*، و *PoET* ساختار رقابت را تغییر داده و رفتار خودخواهانه را کمتر سودآور می‌کنند.
۵. قرعه‌کشی برای انتخاب بلاک‌ساز: در پروتکل‌هایی مثل *Algorand*، انتخاب ماینرها برای تولید بلاک به صورت رمزنگاری شده و تصادفی است. این کار باعث کاهش تأثیر مستقیم قدرت پردازشی می‌شود.
۶. شکستن تساوی به صورت تصادفی: در مواقعی که چند زنجیره با طول برابر وجود دارد، ماینرها به صورت تصادفی یکی را انتخاب می‌کنند. این امر استخراج خودخواهانه را غیرقابل پیش‌بینی و کم‌سود می‌کند.
۷. برچسب زمانی (*Timestamping*): الزام به داشتن برچسب زمانی قابل اعتماد برای بلاک‌ها باعث می‌شود ماینرهایی که بلاک را نگه داشته‌اند و دیر منتشر می‌کنند، جریمه شوند.
۸. بلاک‌های ساختگی (*Dummy Blocks*): در صورتی که ماینر نتواند بلاک واقعی پیدا کند، موظف است بلاک ساختگی منتشر کند. این کار نشان‌دهنده مشارکت صادقانه اوست و جلوی ایجاد زنجیره‌های مخفی را می‌گیرد.
۹. استفاده از ساختارهای *BlockDAG*: پروتکل‌هایی مثل *SPECTRE*، *GHOST*، و *PHANTOM* با تغییر ساختار زنجیره به گراف غیرمدور (*DAG*)، مزایای استخراج خودخواهانه را کاهش داده و امکان پذیرش بلاک‌های موازی را فراهم می‌کنند.

مراجع

- [1] “fake stake attacks,” <https://medium.com/@dsl-uiuc/fake-stake-attacks-on-chain-based-proof-of-stake-cryptocurrencies>, 2019.
- [2] “Iota miota whitepaper,” <https://IOTA-MIOTA-Whitepaper.pdf> , 2018.
- [3] “tangle weighted random walk,” <https://www.blockchainguide.biz/tangle-weighted-random-walk/>, 2018.